LAPORAN

PRAKTIKUM DASAR PEMROGRAMAN

JOBSHEET 7

PERULANGAN 1



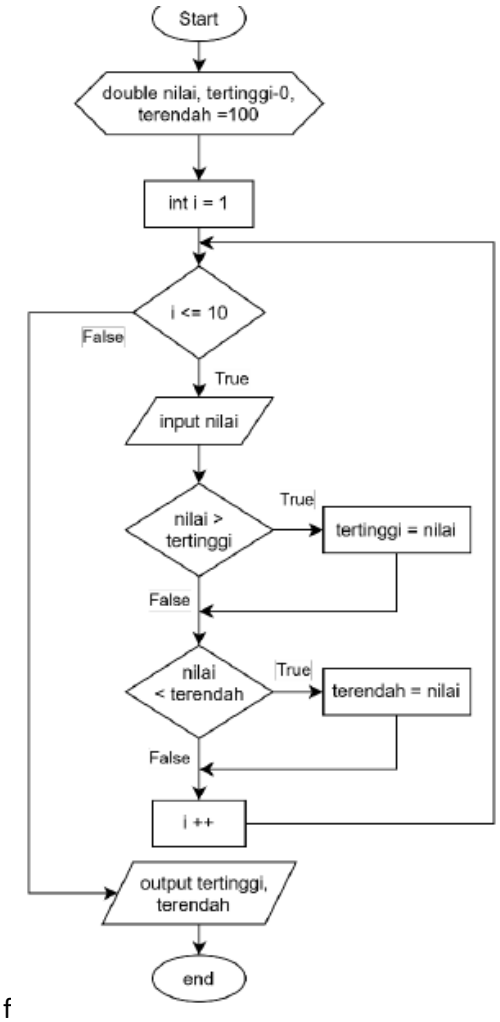
Nama Lengkap	:	Arya Maulana
NIM.	:	264107020194
Program Studi	:	Sarjana Terapan Teknik Informatika
Kelas / Absen	:	TI – 1B / 06

JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG
2026

Percobaan 1: Studi Kasus Nilai Mahasiswa di SIAKAD – Perulangan FOR

STUDI KASUS

Di Sistem Informasi Akademik (SIAKAD), dosen mengisi nilai mata kuliah Praktikum Dasar Pemrograman yang ditempuh oleh mahasiswa. Dosen ingin mencari nilai tertinggi dan terendah Kuis dari 10 mahasiswa di dalam satu kelas. Dosen harus memasukkan nilai dari setiap mahasiswa, kemudian program menentukan dan menampilkan nilai tertinggi dan terendah. Program Java dibuat berdasarkan flowchart pada jobsheet.



f

Gambar 1. Flowchart Percobaan 1 (Perulangan FOR)

A. *Tahapan Percobaan yang Dilakukan*

1. Membuat repository baru pada akun GitHub dengan nama **daspro-jobsheet7**.
2. Melakukan cloning repository tersebut menggunakan perintah `git clone` dari terminal.
3. Membuka folder repository hasil cloning menggunakan Visual Studio Code.
4. Membuat file baru dengan nama **SiakadFor06.java** (06 adalah nomor absen).
5. Membuat struktur dasar program Java yang terdiri dari fungsi `main()`, serta menambahkan library Scanner di bagian atas (luar) class.

```
import java.util.Scanner;

public class SiakadFor06 {
    public static void main(String[] args) {

    }
}
```

6. Membuat deklarasi Scanner dengan nama variabel `input` di dalam fungsi `main()`.

```
Scanner input = new Scanner(System.in);
```

7. Mendeklarasikan variabel `nilai`, `tertinggi`, dan `terendah` bertipe double. Variabel `tertinggi` diinisialisasi dengan 0 dan `terendah` dengan 100, sesuai blok deklarasi pada flowchart.

```
double nilai, tertinggi = 0, terendah = 100;
```

8. Membuat struktur perulangan FOR dengan batas kondisi sesuai jumlah mahasiswa, yaitu 10.

```
for (int i = 1; i <= 10; i++) {

}
```

9. Di dalam perulangan FOR, menambahkan perintah untuk memasukkan nilai mahasiswa, kemudian membuat dua kondisi pemilihan IF secara terpisah untuk membandingkan `nilai` dengan `tertinggi` dan `terendah`.

```
for (int i = 1; i <= 10; i++) {
    System.out.print("Masukkan nilai mahasiswa ke-" + i + ": ");
    nilai = input.nextDouble();
    if (nilai > tertinggi) {
        tertinggi = nilai;
    }
    if (nilai < terendah) {
        terendah = nilai;
    }
}
```

10. Di luar perulangan FOR, menampilkan nilai tertinggi dan terendah.

```
System.out.println("Nilai tertinggi: " + tertinggi);
System.out.println("Nilai terendah: " + terendah);
```

11. Melakukan compile dan run program, lalu mencocokkan hasilnya dengan gambar pada bagian Verifikasi Hasil Percobaan di jobsheet.
12. Melakukan Add, Commit, dan Push kode program ke GitHub.

B. ***Kode Program***

```
import java.util.Scanner;

public class SiakadFor06 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        double nilai, tertinggi = 0, terendah = 100;

        for (int i = 1; i <= 10; i++) {
            System.out.print("Masukkan nilai mahasiswa ke-" + i + ": ");
            nilai = input.nextDouble();
            if (nilai > tertinggi) {
                tertinggi = nilai;
            }
            if (nilai < terendah) {
                terendah = nilai;
            }
        }

        System.out.println("Nilai tertinggi: " + tertinggi);
        System.out.println("Nilai terendah: " + terendah);
    }
}
```

C. ***Bukti Running / Output***

```
Masukkan nilai mahasiswa ke-1: 76.5
Masukkan nilai mahasiswa ke-2: 82.3
Masukkan nilai mahasiswa ke-3: 62.1
Masukkan nilai mahasiswa ke-4: 88.4
Masukkan nilai mahasiswa ke-5: 65.9
Masukkan nilai mahasiswa ke-6: 67.9
Masukkan nilai mahasiswa ke-7: 90.1
Masukkan nilai mahasiswa ke-8: 55.3
Masukkan nilai mahasiswa ke-9: 73.7
Masukkan nilai mahasiswa ke-10: 78.6
Nilai tertinggi: 90.1
Nilai terendah: 55.3
```

D. *Pertanyaan*

1. Sebutkan dan tunjukkan masing-masing komponen perulangan FOR pada kode program Percobaan 1!

Jawab: Perulangan FOR pada program berbentuk for (int i = 1; i <= 10; i++) { ... } dan memiliki empat komponen:

Komponen	Bagian pada kode	Keterangan
Inisialisasi	`int i = 1`	Dijalankan satu kali di awal; membuat variabel penghitung `i` dengan nilai awal 1 (mahasiswa pertama).
Kondisi	`i <= 10`	Diperiksa sebelum setiap putaran; selama bernilai true badan perulangan dijalankan, jika false perulangan berhenti.
Update	`i++`	Dijalankan di akhir setiap putaran untuk menaikkan `i` sebesar 1.
Badan perulangan	Blok `{ ... }` berisi input nilai dan dua IF	Perintah yang diulang sebanyak 10 kali, yaitu memasukkan nilai lalu memperbarui tertinggi dan terendah.

2. Mengapa variabel tertinggi diinisialisasi 0 dan terendah diinisialisasi 100? Apa yang terjadi jika variabel tertinggi diinisialisasi 100 dan terendah diinisialisasi 0?

Jawab: Inisialisasi dilakukan pada batas ekstrem agar input pertama langsung menggantikannya: tertinggi diset 0 agar nilai berapa pun (≥ 0) pasti lebih besar, dan terendah diset 100 agar nilai berapa pun (≤ 100) pasti lebih kecil. Jika dibalik (tertinggi = 100, terendah = 0), tidak ada nilai input yang bisa memicu perubahan variabel karena tidak ada nilai > 100 atau < 0, sehingga program selalu salah menampilkan hasil akhir 100 dan 0.

3. Jelaskan fungsi dan alur kerja dari potongan kode berikut!

```
if (nilai > tertinggi) {
    tertinggi = nilai;
}
if (nilai < terendah) {
    terendah = nilai;
}
```

Kode ini berfungsi memperbarui nilai maksimum dan minimum dari data yang dimasukkan. Alur kerjanya membandingkan variabel nilai pada setiap putaran: jika lebih besar dari tertinggi, maka tertinggi diperbarui; jika lebih kecil dari tertinggi (maksudnya terendah), maka terendah diperbarui. Kedua kondisi digolongkan sebagai dua pernyataan if terpisah, bukan if-else, agar keduanya selalu diperiksa, sehingga pada input awal satu nilai yang sama bisa langsung menjadi nilai tertinggi sekaligus terendah.

4. Modifikasi kode program sehingga terdapat perhitungan untuk menentukan berapa mahasiswa yang lulus dan yang tidak lulus berdasarkan batas kelulusan (nilai minimal 60). Tampilkan jumlah mahasiswa lulus dan tidak lulus setelah menampilkan nilai tertinggi dan terendah!

Jawab: Ditambahkan dua variabel penghitung lulus dan tidakLulus yang bernilai awal 0. Di dalam perulangan, nilai diperiksa dengan batas kelulusan 60: jika nilai >= 60 maka lulus bertambah,

selain itu tidakLulus bertambah. Jumlah keduanya ditampilkan setelah nilai tertinggi dan terendah. Berikut bagian yang diubah (file **SiakadFor06.java** setelah dimodifikasi):

```
import java.util.Scanner;

public class SiakadFor06 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        double nilai, tertinggi = 0, terendah = 100;
        int lulus = 0, tidakLulus = 0;

        for (int i = 1; i <= 10; i++) {
            System.out.print("Masukkan nilai mahasiswa ke-" + i + ": ");
            nilai = input.nextDouble();
            if (nilai > tertinggi) {
                tertinggi = nilai;
            }
            if (nilai < terendah) {
                terendah = nilai;
            }
            if (nilai >= 60) {
                lulus++;
            } else {
                tidakLulus++;
            }
        }

        System.out.println("Nilai tertinggi: " + tertinggi);
        System.out.println("Nilai terendah: " + terendah);
        System.out.println("Jumlah mahasiswa lulus: " + lulus);
        System.out.println("Jumlah mahasiswa tidak lulus: " + tidakLulus);
    }
}
```

Hasil run program setelah dimodifikasi

```
Masukkan nilai mahasiswa ke-1: 76.5
Masukkan nilai mahasiswa ke-2: 82.3
Masukkan nilai mahasiswa ke-3: 62.1
Masukkan nilai mahasiswa ke-4: 88.4
Masukkan nilai mahasiswa ke-5: 65.9
Masukkan nilai mahasiswa ke-6: 67.9
Masukkan nilai mahasiswa ke-7: 90.1
Masukkan nilai mahasiswa ke-8: 55.3
Masukkan nilai mahasiswa ke-9: 73.7
Masukkan nilai mahasiswa ke-10: 78.6
Nilai tertinggi: 90.1
Nilai terendah: 55.3
Jumlah mahasiswa lulus: 9
Jumlah mahasiswa tidak lulus: 1
```

Percobaan 2: Studi Kasus Nilai Mahasiswa di SIAKAD – Perulangan WHILE

STUDI KASUS

Seorang dosen ingin memasukkan nilai beberapa mahasiswa ke dalam SIAKAD untuk ditentukan kategori nilai hurufnya. Program meminta dosen memasukkan nilai setiap mahasiswa. Jika nilai yang dimasukkan tidak valid (negatif atau lebih dari 100), program mengabaikan input tersebut dan meminta dosen melakukan input ulang. Nilai yang valid kemudian dikelompokkan ke dalam kategori huruf berikut:

Huruf	A	B+	B	C+	C	D	E
Rentang nilai	80 < nilai ≤ 100	73 < nilai ≤ 80	65 < nilai ≤ 73	60 < nilai ≤ 65	50 < nilai ≤ 60	39 < nilai ≤ 50	nilai ≤ 39

A. *Tahapan Percobaan yang Dilakukan*

1. Membuat file baru dengan nama **SiakadWhile06.java** di dalam repository daspro-jobsheet7.
2. Membuat struktur dasar program Java yang terdiri dari fungsi main(), menambahkan library Scanner di bagian atas (luar) class, dan membuat deklarasi Scanner dengan nama variable input di dalam main().
3. Mendeklarasikan variabel nilai, jml, dan i (untuk perulangan) bertipe integer. Variabel i diinisialisasi dengan 0 sebagai nilai awal perulangan.

```
int nilai, jml, i = 0;
```

4. Menuliskan kode program untuk menerima input banyaknya mahasiswa yang disimpan ke variabel `jml`, sehingga batas perulangan bersifat dinamis sesuai masukan pengguna.

```
System.out.print("Masukkan jumlah mahasiswa: ");
jml = input.nextInt();
```

5. Membuat struktur perulangan WHILE dengan kondisi i < jml. Simbol yang dipakai adalah < karena variabel i dimulai dari 0, bukan 1. Pernyataan i++ diletakkan di bagian akhir badan perulangan.

```
while (i < jml) {

    i++;
}
```

6. Di dalam perulangan WHILE, menambahkan perintah untuk memasukkan nilai mahasiswa. Kemudian membuat kondisi IF untuk memeriksa valid atau tidaknya nilai (harus pada rentang 0 sampai 100); jika tidak valid ditampilkan pesan dan perintah continue dijalankan.

```
System.out.print("Masukkan nilai mahasiswa ke-" + (i + 1) + ": ");
nilai = input.nextInt();

if (nilai < 0 || nilai > 100) {
    System.out.println("Nilai tidak valid. Masukkan lagi nilai yang valid!");
    continue;
}
```

7. Menambahkan kondisi pemilihan IF - ELSE IF - ELSE untuk menampilkan kategori nilai huruf (A, B+, B, C+, C, D, dan E) berdasarkan ketentuan pada studi kasus. Struktur lengkapnya dapat dilihat pada bagian Kode Program.
8. Melakukan compile dan run program, lalu mencocokkan hasilnya dengan gambar pada jobsheet.
9. Melakukan Add, Commit, dan Push kode program ke GitHub.

B. *Kode Program*

```
import java.util.Scanner;

public class SiakadWhile06 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        int nilai, jml, i = 0;

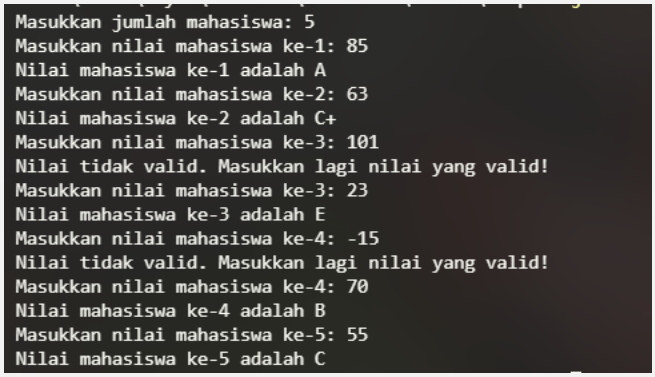
        System.out.print("Masukkan jumlah mahasiswa: ");
        jml = input.nextInt();

        while (i < jml) {
            System.out.print("Masukkan nilai mahasiswa ke-" + (i + 1) + ": ");
            nilai = input.nextInt();

            if (nilai < 0 || nilai > 100) {
                System.out.println("Nilai tidak valid. Masukkan lagi nilai yang valid!");
                continue;
            }

            if (nilai > 80 && nilai <= 100) {
                System.out.println("Nilai mahasiswa ke-" + (i + 1) + " adalah A");
            } else if (nilai > 73 && nilai <= 80) {
                System.out.println("Nilai mahasiswa ke-" + (i + 1) + " adalah B+");
            } else if (nilai > 65 && nilai <= 73) {
                System.out.println("Nilai mahasiswa ke-" + (i + 1) + " adalah B");
            } else if (nilai > 60 && nilai <= 65) {
                System.out.println("Nilai mahasiswa ke-" + (i + 1) + " adalah C+");
            } else if (nilai > 50 && nilai <= 60) {
                System.out.println("Nilai mahasiswa ke-" + (i + 1) + " adalah C");
            } else if (nilai > 39 && nilai <= 50) {
                System.out.println("Nilai mahasiswa ke-" + (i + 1) + " adalah D");
            } else {
                System.out.println("Nilai mahasiswa ke-" + (i + 1) + " adalah E");
            }
            i++;
        }
    }
}
```

C. *Bukti Running / Output*



Hasil pengujian:

Uji	Nilai masukan	Keluaran	Keterangan
1	85	A	Valid
1	63	C+	Valid
1	101	Nilai tidak valid. Masukkan lagi nilai yang valid!	Ditolak, input diulang untuk mahasiswa yang sama
1	23	E	Valid
1	-15	Nilai tidak valid. Masukkan lagi nilai yang valid!	Ditolak, input diulang untuk mahasiswa yang sama
1	70	B	Valid
1	55	C	Valid
2	100	A	Batas atas
2	80	B+	80 termasuk B+ (73 < nilai ≤ 80)
2	73	B	73 termasuk B
2	65	C+	65 termasuk C+
2	60	C	60 termasuk C
2	50	D	50 termasuk D
2	39	E	39 termasuk E
2	0	E	Batas bawah

D. *Jawaban Pertanyaan*

1. Pada potongan kode berikut, tentukan maksud dan kegunaan dari sintaks berikut:

```
if (nilai < 0 || nilai > 100) {
    System.out.println("Nilai tidak valid. Masukkan lagi nilai yang valid!");
    continue;
}
```

a. nilai < 0 || nilai > 100

Jawab: Operator || (OR) menghasilkan true apabila salah satu kondisi terpenuhi. Ekspresi ini bernilai true jika nilai negatif (nilai < 0) atau lebih dari 100 (nilai > 100), yaitu nilai yang berada di luar rentang valid 0 sampai 100. Kegunaannya sebagai validasi input sebelum nilai dikelompokkan ke dalam kategori huruf.

b. continue

Jawab: Perintah continue menghentikan putaran yang sedang berjalan dan langsung kembali ke pemeriksaan kondisi while (i < jml), sehingga sisa kode di bawahnya (penentuan huruf dan i++) dilewati. Karena i tidak bertambah, program kembali meminta nilai untuk mahasiswa yang sama.

2. Mengapa sintaks i++ dituliskan di akhir perulangan WHILE? Apa yang terjadi jika posisinya dituliskan di awal perulangan WHILE?

Jawab: Sintaks i++ diletakkan di akhir agar variabel hanya bertambah setelah input dinyatakan valid. Jika terjadi kesalahan input, perintah continue akan melompati i++ sehingga penomoran tidak bertambah dan pengguna diminta mengulang input yang sama.

Jika ditulis di awal, nilai i akan bertambah sebelum validasi dilakukan. Akibatnya, input yang salah tetap menghitung jatah putaran (penomoran bergeser) sehingga data yang valid menjadi kurang dari target, dan penomoran tampilan pesan pun ikut salah.

3. Apabila jumlah mahasiswa yang dimasukkan adalah 19, berapa kali perulangan WHILE akan berjalan?

Jawab: Variabel i dimulai dari 0 dan perulangan berjalan selama i < 19, sehingga nilai i adalah 0, 1, 2, ..., 18. Perulangan WHILE berjalan **19 kali** untuk 19 nilai yang valid. Jika ada input tidak valid, setiap input tersebut menambah satu putaran ekstra karena continue tidak menaikkan i, sehingga jumlah putaran sebenarnya menjadi 19 ditambah banyaknya input yang ditolak.

4. Modifikasi kode program sehingga apabila terdapat mahasiswa yang mendapat nilai A, program menampilkan pesan tambahan "Bagus, pertahankan nilainya"!

Pada cabang if untuk nilai A ditambahkan satu perintah System.out.println yang menampilkan pesan "Bagus, pertahankan nilainya".

```
package Jobsheet7;

import java.util.Scanner;

public class ModifikasiSiakadWhile06 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        int nilai, jml, i = 0;

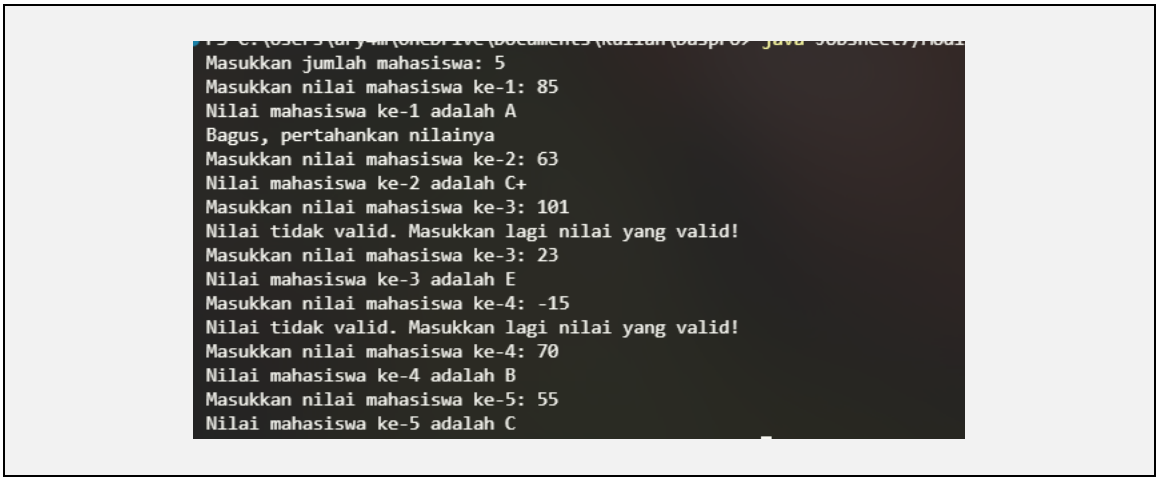
        System.out.print("Masukkan jumlah mahasiswa: ");
        jml = input.nextInt();

        while (i < jml) {
            System.out.print("Masukkan nilai mahasiswa ke-" + (i + 1) + ": 
");
            nilai = input.nextInt();

            if (nilai < 0 || nilai > 100) {
                System.out.println("Nilai tidak valid. Masukkan lagi nilai 
yang valid!");
                continue;
            }

            if (nilai > 80 && nilai <= 100) {
                System.out.println("Nilai mahasiswa ke-" + (i + 1) + " adalah 
A");
                System.out.println("Bagus, pertahankan nilainya"); // 
tambahan
            } else if (nilai > 73 && nilai <= 80) {
                System.out.println("Nilai mahasiswa ke-" + (i + 1) + " adalah 
B+");
            } else if (nilai > 65 && nilai <= 73) {
                System.out.println("Nilai mahasiswa ke-" + (i + 1) + " adalah 
B");
            } else if (nilai > 60 && nilai <= 65) {
                System.out.println("Nilai mahasiswa ke-" + (i + 1) + " adalah 
C+");
            } else if (nilai > 50 && nilai <= 60) {
                System.out.println("Nilai mahasiswa ke-" + (i + 1) + " adalah 
C");
            } else if (nilai > 39 && nilai <= 50) {
                System.out.println("Nilai mahasiswa ke-" + (i + 1) + " adalah 
D");
            } else {
                System.out.println("Nilai mahasiswa ke-" + (i + 1) + " adalah 
E");
            }
            i++;
        }
    }
}
```

Hasil run program setelah dimodifikasi



Percobaan 3: Studi Kasus Transaksi di Kafe – Perulangan DO-WHILE

STUDI KASUS

Di sebuah kafe, kasir ingin memproses transaksi beberapa pelanggan. Pelanggan dapat membeli lebih dari satu item (kopi seharga Rp 12.000, teh seharga Rp 7.000, dan roti seharga Rp 20.000), dan kasir terus memasukkan jumlah pembelian untuk setiap pelanggan. Jika ada pelanggan yang memutuskan membatalkan transaksi (dengan memasukkan "batal"), kasir menghentikan input transaksi dan program berhenti.

A. *Tahapan Percobaan yang Dilakukan*

1. Membuat file baru dengan nama **KafeDoWhile06.java** di dalam repository daspro-jobsheet7.
2. Membuat struktur dasar program Java yang terdiri dari fungsi main(), menambahkan library Scanner di bagian atas (luar) class, dan membuat deklarasi Scanner dengan nama variabel input di dalam main().
3. Mendeklarasikan variabel kopi, teh, dan roti bertipe integer untuk menampung banyaknya item yang dibeli, totalHarga bertipe integer untuk menampung total pembayaran, serta namaPelanggan bertipe String. Mendeklarasikan sekaligus menginisialisasi hargaKopi dengan 12000, hargaTeh dengan 7000, dan hargaRoti dengan 20000.

```
int kopi, teh, roti, totalHarga;
String namaPelanggan;
int hargaKopi = 12000;
int hargaTeh = 7000;
int hargaRoti = 20000;
```

4. Membuat struktur perulangan DO-WHILE dengan kondisi true.

```
do {

} while (true);
```

5. Di dalam perulangan, menambahkan perintah untuk memasukkan namaPelanggan, kemudian kondisi IF untuk memeriksa isinya. Jika nama sama dengan "batal" (tanpa memperhatikan huruf besar/kecil) ditampilkan pesan pembatalan lalu perulangan dihentikan dengan break.

```
System.out.print("Masukkan nama pelanggan (ketik 'batal' untuk keluar): ");
namaPelanggan = input.nextLine();
if (namaPelanggan.equalsIgnoreCase("batal")) {
    System.out.println("Transaksi dibatalkan.");
    break;
}
```

6. Apabila nama pelanggan bukan "batal", menambahkan perintah untuk memasukkan banyaknya kopi, teh, dan roti yang dibeli, menghitung total harga, dan menampilkannya. Perintah input.nextLine(); di akhir badan perulangan berfungsi membersihkan karakter newline yang tersisa di buffer setelah nextInt().

```
totalHarga = (kopi * hargaKopi) + (teh * hargaTeh) + (roti * hargaRoti);
System.out.println("Total yang harus dibayar: Rp " + totalHarga);
input.nextLine();
```

7. Di luar perulangan, menampilkan pesan bahwa semua transaksi selesai.
8. Melakukan compile dan run program, lalu mencocokkan hasilnya dengan gambar pada jobsheet.
9. Melakukan Add, Commit, dan Push kode program ke GitHub.

B. *Kode Program*

```
import java.util.Scanner;

public class KafeDoWhile06 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        int kopi, teh, roti, totalHarga;
        String namaPelanggan;
        int hargaKopi = 12000;
        int hargaTeh = 7000;
        int hargaRoti = 20000;

        do {
            System.out.print("Masukkan nama pelanggan (ketik 'batal' untuk keluar): ");
            namaPelanggan = input.nextLine();
            if (namaPelanggan.equalsIgnoreCase("batal")) {
                System.out.println("Transaksi dibatalkan.");
                break;
            }
            System.out.print("Jumlah kopi: ");
            kopi = input.nextInt();
            System.out.print("Jumlah teh: ");
            teh = input.nextInt();
            System.out.print("Jumlah roti: ");
            roti = input.nextInt();

            totalHarga = (kopi * hargaKopi) + (teh * hargaTeh) + (roti * hargaRoti);
            System.out.println("Total yang harus dibayar: Rp " + totalHarga);
            input.nextLine();
        } while (true);

        System.out.println("Semua transaksi selesai.");
    }

}
```

C. *Bukti Running / Output*

```
Masukkan nama pelanggan (ketik 'batal' untuk keluar): Rena
Jumlah kopi: 3
Jumlah teh: 0
Jumlah roti: 1
Total yang harus dibayar: Rp 56000
Masukkan nama pelanggan (ketik 'batal' untuk keluar): Yuni
Jumlah kopi: 1
Jumlah teh: 4
Jumlah roti: 2
Total yang harus dibayar: Rp 80000
Masukkan nama pelanggan (ketik 'batal' untuk keluar): BATAL
Transaksi dibatalkan.
Semua transaksi selesai.
```

Hasil pengujian:

Uji	Pelanggan	Kopi	Teh	Roti	Perhitungan	Total
1	Rena	3	0	1	3×12000 + 0×7000 + 1×20000	Rp 56000
1	Yuni	1	4	2	1×12000 + 4×7000 + 2×20000	Rp 80000
1	BATAL	-	-	-	Transaksi dibatalkan	-
2	batal	-	-	-	Transaksi dibatalkan (iterasi pertama)	-

D. *Jawaban Pertanyaan*

1. Pada penggunaan DO-WHILE ini, apabila nama pelanggan yang dimasukkan pertama kali adalah "batal", maka berapa kali perulangan dilakukan?

Jawab: Perulangan dilakukan **1 kali**. Pada DO-WHILE, badan perulangan dijalankan terlebih dahulu sebelum kondisi diperiksa. Pada putaran pertama nama "batal" dimasukkan, kondisi IF terpenuhi, pesan pembatalan ditampilkan, dan `break` langsung menghentikan perulangan (lihat Uji 2).

2. Sebutkan kondisi berhenti yang digunakan pada perulangan DO-WHILE tersebut!

Jawab: Kondisi while (true) tidak pernah bernilai false, sehingga perulangan tidak berhenti karena kondisi tersebut. Perulangan berhenti ketika namaPelanggan.equalsIgnoreCase("batal") bernilai true, yaitu kasir mengetik "batal" (huruf besar/kecil tidak berpengaruh), dan perintah break dijalankan.

3. Apa fungsi dari penggunaan nilai true pada kondisi DO-WHILE?

Jawab: Nilai true membuat kondisi perulangan selalu terpenuhi sehingga perulangan terus berulang tanpa batas jumlah putaran tertentu. Cara ini dipakai karena banyaknya pelanggan tidak diketahui di awal. Perulangan baru berhenti jika ada break, yang pada program ini dipicu oleh input "batal".

4. Mengapa perulangan DO-WHILE tersebut tetap berjalan meskipun tidak ada komponen inisialisasi dan update?

Jawab: Komponen inisialisasi dan update hanya diperlukan jika perulangan dikendalikan oleh variabel penghitung (seperti i pada FOR atau WHILE dengan batas jumlah). Perulangan DO-WHILE ini tidak dikendalikan oleh penghitung, melainkan oleh kondisi true dan input dari pengguna. Jumlah putarannya tidak ditentukan oleh nilai variabel yang dinaikkan, tetapi oleh kapan kasir memasukkan "batal", sehingga tidak ada variabel yang perlu diinisialisasi maupun di-update.

Tugas 1: Penjualan Tiket Bioskop dalam Satu Hari

SOAL

Pengelola bioskop ingin membuat program untuk menghitung total penjualan tiket dalam satu hari. Harga tiket Rp 50.000 per tiket. Program menghitung total tiket yang terjual dan total harga penjualan tiket selama satu hari dengan ketentuan: pembelian lebih dari 4 tiket mendapat diskon 10%; pembelian lebih dari 10 tiket mendapat diskon 15%; dan jika input jumlah tiket tidak valid (negatif), program mengabaikan input tersebut dan meminta input ulang.

Karena soal tidak menyebutkan cara mengakhiri input, program memakai input **0** sebagai tanda tidak ada lagi pelanggan (akhir hari).

A. *Tahapan Percobaan yang Dilakukan*

1. Membuat file baru dengan nama **BioskopTiket06.java** di dalam repository daspro-jobsheet7, lalu membuat struktur dasar program dan Scanner input.
2. Mendeklarasikan hargaTiket dengan nilai 50000, variabel jumlahTiket, totalHarga, diskon, dan totalBayar bertipe integer, serta dua variabel akumulator totalTiketTerjual dan totalPenjualan yang diinisialisasi 0.
3. Memilih perulangan DO-WHILE dengan kondisi true karena jumlah pelanggan dalam satu hari tidak diketahui di awal; perulangan dihentikan dengan break.
4. Di dalam perulangan, meminta input jumlahTiket. Jika jumlahTiket < 0 ditampilkan pesan tidak valid dan continue dijalankan agar input diulang. Jika jumlahTiket == 0 perulangan dihentikan dengan break.
5. Menghitung totalHarga = jumlahTiket * hargaTiket, kemudian menentukan diskon dengan IF - ELSE IF. Syarat lebih dari 10 tiket diperiksa lebih dulu karena nilai yang lebih dari 10 pasti juga lebih dari 4.

```
diskon = 0;
if (jumlahTiket > 10) {
    diskon = totalHarga * 15 / 100;
} else if (jumlahTiket > 4) {
    diskon = totalHarga * 10 / 100;
}
totalBayar = totalHarga - diskon;
```

6. Menampilkan rincian transaksi, menambahkan jumlahTiket ke totalTiketTerjual dan totalBayar ke totalPenjualan. Setelah perulangan selesai, menampilkan kedua total tersebut.
7. Melakukan compile dan run program dengan beberapa data uji, kemudian Add, Commit, dan Push ke GitHub.

B. *Kode Program*

```
import java.util.Scanner;

public class BioskopTiket06 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        int hargaTiket = 50000;
        int jumlahTiket, totalHarga, diskon, totalBayar;
        int totalTiketTerjual = 0;
        int totalPenjualan = 0;

        do {
            System.out.print("Masukkan jumlah tiket (0 untuk selesai): ");
            jumlahTiket = input.nextInt();

            if (jumlahTiket < 0) {
                System.out.println("Jumlah tiket tidak valid. Masukkan lagi!");
                continue;
            }
            if (jumlahTiket == 0) {
                break;
            }

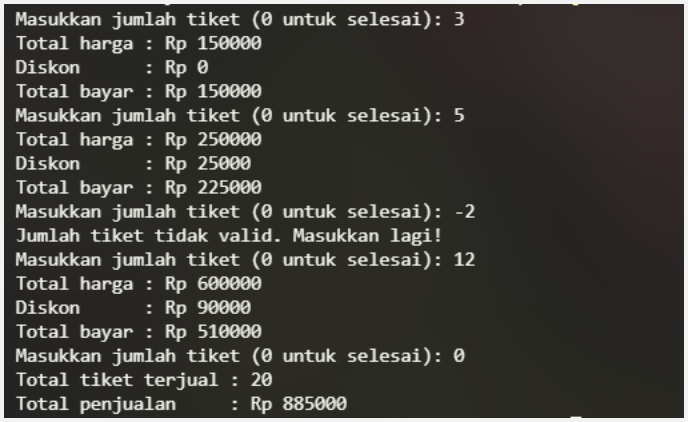
            totalHarga = jumlahTiket * hargaTiket;
            diskon = 0;
            if (jumlahTiket > 10) {
                diskon = totalHarga * 15 / 100;
            } else if (jumlahTiket > 4) {
                diskon = totalHarga * 10 / 100;
            }
            totalBayar = totalHarga - diskon;

            System.out.println("Total harga : Rp " + totalHarga);
            System.out.println("Diskon      : Rp " + diskon);
            System.out.println("Total bayar : Rp " + totalBayar);

            totalTiketTerjual += jumlahTiket;
            totalPenjualan += totalBayar;
        } while (true);

        System.out.println("Total tiket terjual : " + totalTiketTerjual);
        System.out.println("Total penjualan     : Rp " + totalPenjualan);
    }
}
```

C. *Bukti Running / Output*



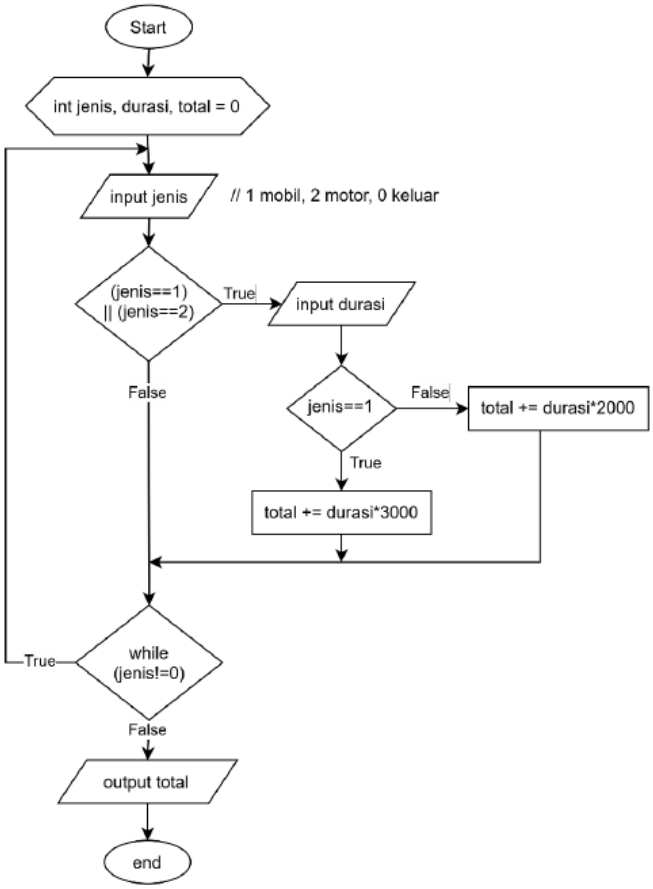
Hasil pengujian:

Uji	Jumlah tiket	Total harga	Diskon	Total bayar	Keterangan
1	3	Rp 150000	Rp 0	Rp 150000	Tidak ada diskon
1	5	Rp 250000	Rp 25000	Rp 225000	Diskon 10% (> 4 tiket)
1	-2	-	-	-	Tidak valid, input diulang
1	12	Rp 600000	Rp 90000	Rp 510000	Diskon 15% (> 10 tiket)
1	0	-	-	-	Selesai: 20 tiket, penjualan Rp 885000
2	4	Rp 200000	Rp 0	Rp 200000	Batas: 4 tiket belum diskon
2	10	Rp 500000	Rp 50000	Rp 450000	Batas: 10 tiket masih diskon 10%
2	11	Rp 550000	Rp 82500	Rp 467500	Batas: 11 tiket diskon 15%
2	0	-	-	-	Selesai: 25 tiket, penjualan Rp 1117500

Tugas 2: Implementasi Flowchart Pembayaran Parkir

SOAL

Sebuah tempat parkir ingin membuat program untuk menghitung total pembayaran parkir dari beberapa kendaraan. Tarif parkir adalah Rp 3.000 per jam untuk mobil dan Rp 2.000 per jam untuk motor. Program terus meminta masukan selama input bukan 0. Flowchart tersebut diimplementasikan ke dalam program Java.



Gambar 2. Flowchart Tugas 2 (Pembayaran Parkir)

A. *Tahapan Percobaan yang Dilakukan*

1. Membuat file baru dengan nama **Parkir06.java** di dalam repository daspro-jobsheet7, lalu membuat struktur dasar program dan Scanner input.
2. Mengimplementasikan blok deklarasi pada flowchart: int jenis, durasi, total = 0;.
3. Mengimplementasikan alur flowchart dengan perulangan DO-WHILE, karena input jenis kendaraan harus diminta minimal satu kali dan kondisi jenis != 0 diperiksa di akhir (sesuai simbol while di bagian bawah flowchart).
4. Di dalam perulangan: input jenis; jika jenis == 1 || jenis == 2 maka input durasi; jika jenis == 1 (mobil) total += durasi * 3000, selain itu (motor) total += durasi * 2000.

```
if (jenis == 1 || jenis == 2) {
    System.out.print("Masukkan durasi parkir (jam): ");
    durasi = input.nextInt();
    if (jenis == 1) {
        total += durasi * 3000;
    } else {
        total += durasi * 2000;
    }
}
```

5. Setelah perulangan berhenti (input 0), menampilkan total. Jenis kendaraan selain 0, 1, dan 2 diabaikan sesuai flowchart karena tidak masuk ke percabangan.
6. Melakukan compile dan run program, kemudian Add, Commit, dan Push ke GitHub.

B. *Kode Program*

```
import java.util.Scanner;

public class Parkir06 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        int jenis, durasi, total = 0;

        do {
            System.out.print("Masukkan jenis (1 = mobil, 2 = motor, 0 = keluar): ");
            jenis = input.nextInt();

            if (jenis == 1 || jenis == 2) {
                System.out.print("Masukkan durasi parkir (jam): ");
                durasi = input.nextInt();
                if (jenis == 1) {
                    total += durasi * 3000;
                } else {
                    total += durasi * 2000;
                }
            }
        } while (jenis != 0);

        System.out.println("Total pembayaran parkir: Rp " + total);
    }

}
```

C. *Bukti Running / Output*

Uji 1: mobil 2 jam, motor 3 jam, jenis tidak dikenal (5), lalu keluar (0)

```
Masukkan jenis (1 = mobil, 2 = motor, 0 = keluar): 1
Masukkan durasi parkir (jam): 2
Masukkan jenis (1 = mobil, 2 = motor, 0 = keluar): 2
Masukkan durasi parkir (jam): 3
Masukkan jenis (1 = mobil, 2 = motor, 0 = keluar): 5
Masukkan jenis (1 = mobil, 2 = motor, 0 = keluar): 0
Total pembayaran parkir: Rp 12000
```

Hasil pengujian:

Masukan jenis	Masukan durasi	Perhitungan	Total sementara
1 (mobil)	2 jam	2 × 3000	Rp 6000
2 (motor)	3 jam	3 × 2000	Rp 12000
5	-	Diabaikan (bukan 1 atau 2)	Rp 12000
0	-	Perulangan berhenti	Total akhir Rp 12000